

Нормализация данных

Перед нормализацией данных - был проведен этап преданализа на аномалии и особенности структуры данных. На этапе настройки и преданализа использовались **3 parquet-файла**, чтобы снизить ожидание проливки и быстрее вносить изменения.

На основе преданализа были проверены:

- Дубли. Найдено почти 90k из почти 900k строк. Это означает, что при нормализации необходима загрузка order_items через distinct.
- Одна строка сырых данных не равна одной товарной позиции в заказе, требуется декомпозиция и дедупликация.
- Характеристики, относящиеся к заказу, в целом соотносятся с order_id. Единственное, что может расходиться, - это delivery cost, которое изменяется от курьера к курьеру и требует отдельной логики (о которой написали ниже).
- Users (user_id и phone), stores (store_id и address), drivers (driver_id) решено вынести в отдельные таблицы в качестве справочников, также нет аномалий связей в данных функциональных зависимостях.
- Item_id соотносится с title и category, поэтому было решено выделять это в отдельную таблицу products
- Так как есть зависимость от курьера стоимости доставки и на один заказ может приходиться несколько курьеров - решено сделать отдельную таблицу order_drivers.
- При построении DDL были проверены типы данных, чтобы все данные смогли пролиться в таблицы. Текущая спецификация позволяет заливать все данные без потерь из-за разных форматов.
- Найдены аномалии в timestamp параметрах - есть 19 случаев, где дата доставки (delivered_at) меньше даты начала доставки (delivery_started_at)

В качестве целевой формы нормализации была выбрана третья нормальная форма. Ключевые сущности были разделены, транзитивные зависимости убраны, но мы не декомпозировались до подробных справочников и часть полей оставлены в типе text (например, адреса). Распаршивание адресов до городов и прочего - отложили на этап построения витрины, не нормализуя далее.

По итогу было выявлено 7 сущностей и на каждую из них построена таблица - **users, stores, drivers, products, orders, order_items, order_drivers**.

Для вычисления верного delivery cost была использована следующая логика:

- если один курьер на заказе и он доставлен - берем его driver_delivery_cost в orders.delivery_cost_final

- если курьеров несколько и один из них имеет delivered_at - этот курьер считается финальным и его стоимость используем в orders.delivery_cost_final
- если заказ отменен, то orders.delivery_cost_final оставляем null, так как может быть несколько курьеров и стоимость заказа сложно оценить в таком случае

Таблица	Основные параметры
users	user_id, user_phone
stores	store_id, store_address
drivers	driver_id, driver_phone
products	item_id, item_title, item_category
orders	order_id, user_id, store_id, статусы, скидка, delivery_cost_final
order_items (товарные позиции заказа после дедупликации)	order_item_sk, order_id, item_id, quantity, price, order_discount, item_replaced_id
order_drivers (связка заказ-курьер)	order_id, driver_id, driver_delivery_cost, время статусов, is_final_driver

DDL-скрипт

```
create table if not exists raw_orders (
  order_id bigint,
  user_id bigint,
  user_phone text,
  address_text text,
  created_at timestamp,
  paid_at timestamp,
  delivery_started_at timestamp,
  delivered_at timestamp,
  canceled_at timestamp,
  payment_type text,
  item_id bigint,
  item_title text,
  item_category text,
  item_quantity numeric(10,2),
  item_price numeric(10,2),
  item_canceled_quantity numeric(10,2),
  item_replaced_id bigint,
```

```

    order_discount numeric(5,2),
    item_discount numeric(5,2),
    order_cancellation_reason text,
    driver_id bigint,
    driver_phone text,
    delivery_cost numeric(10,2),
    store_id bigint,
    store_address text,
    source_file_name text,
    loaded_at timestamp default now()
);

create table if not exists raw_load_registry (
    file_name text primary key,
    loaded_at timestamp default now()
);

create table if not exists users (
    user_id bigint primary key,
    user_phone text
);

create table if not exists stores (
    store_id bigint primary key,
    store_address text
);

create table if not exists drivers (
    driver_id bigint primary key,
    driver_phone text
);

create table if not exists products (
    item_id bigint primary key,
    item_title text,
    item_category text
);

create table if not exists order_drivers (
    order_id bigint not null,
    driver_id bigint not null references drivers(driver_id),
    driver_delivery_cost numeric(10,2),
    driver_delivered_at timestamp,
    driver_canceled_at timestamp,
    is_final_driver boolean,
    primary key (order_id, driver_id)
);

```

```
create table if not exists orders (  
    order_id bigint primary key,  
    user_id bigint not null references users(user_id),  
    store_id bigint not null references stores(store_id),  
    address_text text,  
    created_at timestamp,  
    paid_at timestamp,  
    delivery_started_at timestamp,  
    delivered_at timestamp,  
    canceled_at timestamp,  
    payment_type text,  
    order_discount numeric(5,2),  
    order_cancellation_reason text,  
    delivery_cost_final numeric(10,2)  
);  
  
create table if not exists order_items (  
    order_item_sk bigserial primary key,  
    order_id bigint not null references orders(order_id),  
    item_id bigint not null references products(item_id),  
    item_quantity numeric(10,2),  
    item_price numeric(10,2),  
    item_canceled_quantity numeric(10,2),  
    item_discount numeric(5,2),  
    item_replaced_id bigint  
);
```